

Python Project – Phase 2 (Design & Incubation) Guidelines

1. Purpose:

Phase 2 focuses on refining the system design before full development begins. Students must break the project into modules, finalize data structures, design functions, and prepare detailed technical planning.

2. Required Submission:

2.1 Refined System Architecture

Students must clearly describe the overall structure of the project, including the list of modules/files, responsibilities of each module, and how modules interact. There should be a clear separation of concerns such as user interface, data handling, and validation.

Example format:

Module Name: main.py

Responsibility: Controls menu and user interaction

Module Name: data_handler.py

Responsibility: Performs file/database operations

The architecture must show modular and organized thinking.

2.2 Detailed Flowcharts (Per Major Function)

Separate flowcharts must be created for each major function such as Add, Update, Delete, Search/View, and Exit. Each flowchart must include input steps, validation, processing, error handling, and output display.

2.3 Function-Level Design

All major functions must be documented with function name, purpose, parameters, return values, and a short description of internal logic. This promotes modular and structured coding.

Example:

Function Name: add_record()

Purpose: Adds new record to storage

Parameters: name (str), amount (float)

Returns: Success/Failure message

This encourages clean and modular coding.

2.4 Final Data Structure / Database Schema

Students must finalize the data storage method (CSV, TXT, JSON, or SQLite). Include field names, data types, constraints, and sample structure before coding begins.

Field Name	Data Type	Constraint
id	INTEGER	PRIMARY KEY
name	TEXT	NOT NULL
amount	REAL	NOT NULL

2.5 UI Mockups or Structured Menu Layout

GUI projects must include screen layout and navigation design. Console projects must include structured menu layouts and user navigation flow. Mockups may be hand-drawn or digitally designed.

2.6 Error Handling Plan

Describe how the system will handle invalid input, incorrect data types, missing files, record not found situations, and other possible exceptions. Mention validation checks and try-except handling.

2.7 Detailed Testing Plan

Prepare structured test cases for each function including normal cases, edge cases, and invalid input scenarios. Testing must be clearly documented.

Function	Test Input	Expected Output
add_record	valid data	Record saved
add_record	empty name	Error message

2.8 Development Timeline (Module-Wise)

Divide development into logical stages such as Menu Setup, Data Storage Implementation, CRUD Operations, and Testing & Debugging. The timeline must be realistic and achievable.

Example:

Week 1 – Menu and Structure

Week 2 – Data Storage Implementation

Week 3 – CRUD Operations

Week 4 – Testing and Debugging